

Unit 1

單元一 軟體開發本質困難原則與標準

台科大資管系軟體工程與管理研究中心
黃世禎教授

Professor Sun-Jen Huang
Software Engineering and Management Lab.
Department of Information Management
National Taiwan University of Science & Technology

學習目標

- 瞭解軟體開發是一項專業技術且需要被管理
- 瞭解軟體開發的四項本質與困難
- 瞭解軟體開發常見的軟體疏失
- 瞭解工程與軟體工程的原則與相關相關理論
- 能夠知道軟體工程包含哪些知識領域
- 瞭解國際與產業軟體工程相關標準

軟體工程

- 1960年代鑑於軟體開發所遭遇到的困境，產生軟體危機，因此北大西洋公約組織於1968年舉辦了首次的軟體工程學術會議
- 軟體工程定義：研究軟體工程原則（Principle）、技術、方法與工具，以提升流程效率、人員生產力與軟體品質
- 軟體工程目標：準時交付（如期）、符合預算（如本）、優異品質（如質）、客戶滿意
- 基於IPO（Input-Process-Output）模型，輸入與流程會是影響輸出品質好壞的主要關鍵因素

軟體開發的IPO模型

- 輸入-流程-輸出 (Input–Process–Output)
- 輸入一般包含有：
 - ▶ 使用者需求(功能性需求、交付時間等)
 - ▶ 軟體開發/管理人員
 - ▶ 技術與軟體工具
- 軟體開發流程
 - ▶ 瀑布式、雛形模型、螺旋模型、敏捷開發、微軟解決方案框架 (Microsoft Solutions Framework, MSF)
 - ▶ 計畫式與敏捷式軟體開發流程
- 輸出一般包含有：
 - ▶ 軟體系統(系統軟體、嵌入式軟體、中介軟體、應用軟體)
 - ▶ 系統文件 (需求規格、系統分析與設計文件、程式碼、測試報告、資安檢測、使用者文件等)

SDLC作業與職稱

作業	工作職稱
規畫與管理	專案/產品經理 (PM)
需求分析/系統分析	系統分析師 (SA)
系統設計	系統設計工程師(SD)、系統架構師 (Architect)、資料庫管理師(DBA)、UI/UX 設計師
程式設計/開發/編碼	程式設計師 (PG)
系統測試、驗證與確認	測試工程師、品質工程師(QA / QC)
部署與維運	DevOps 工程師、維運工程師

何謂優異的軟體系統？

- Functional requirement / 功能性需求
 - Accuracy / 正確
 - Completeness / 完整
- Non-Functional Requirements / 非功能性需求
 - Useability / 系統可用性
 - Performance / 效能
 - Ease of Use / 易用性
 - Reusability / 再使用性
 - Maintainability / 維護性
 - Security / 安全性
 - Portability / 可移植性
 - Etc.

軟體開發的本質

- 從「問題」到「答案」
- 從「人腦」到「電腦」
- 從「程式」到「系統」
- 從「個人」到「團隊」

從「問題」到「答案」

- 尋找**解決**特定的**領域問題**(Domain Problem)的**答案**，並將此解答實作於『軟體系統』，因此軟體開發具備以下的本質：
 - ▶ **複雜性**(complexity)：軟體的目的是要解決問題，其中牽涉到**尋找答案的步驟**，並將答案（**業務流程**）以軟體系統建構出，通常是很**複雜**的。
 - ▶ **專業差異性**：問題領域或使用者的專業與軟體開發專業是不一樣的，因此在**溝通**上也常會**造成問題**。
 - ▶ **變異性**(changeability)：問題領域通常是不是很明確，而且在軟體系統開發過程很有可能會**變動**。
 - ▶ **智能活動**(Mental Activity)：尋找問題的答案與軟體系統的開發都是須要由**人的心智活動**來產出，因此是很容易會**錯誤**的現象

一個悲慘的軟體開發專案故事



專案贊助者的建議



專案的需求規範



資深分析師的設計



程式師的產出



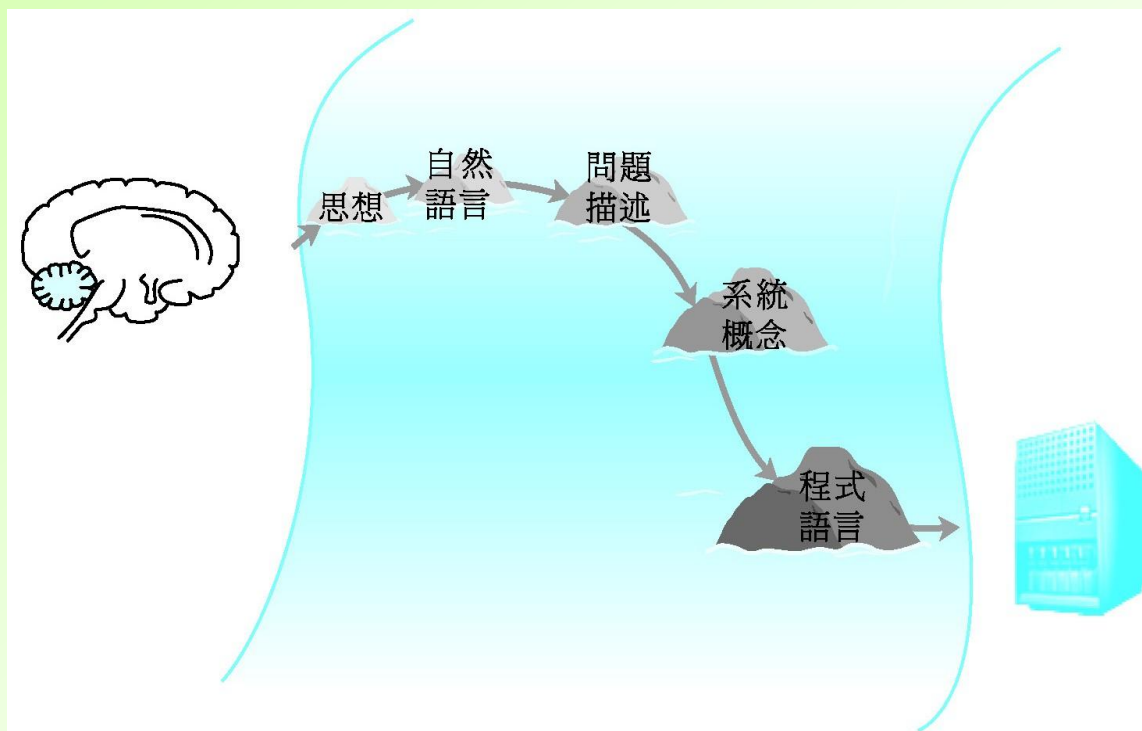
用戶電腦上的安裝



用戶真正的需要

從「人腦」到「電腦」

- 人腦與電腦的差異：人腦與電腦是**不一樣**的，因此須要思考如何將問題的解答（人腦）轉換並執行到軟體系統（電腦）上
- 目前的資訊系統還是**無法完全取代**人腦的思維活動，因此問題的解答需能區分出**人機協同合作的系統流程**



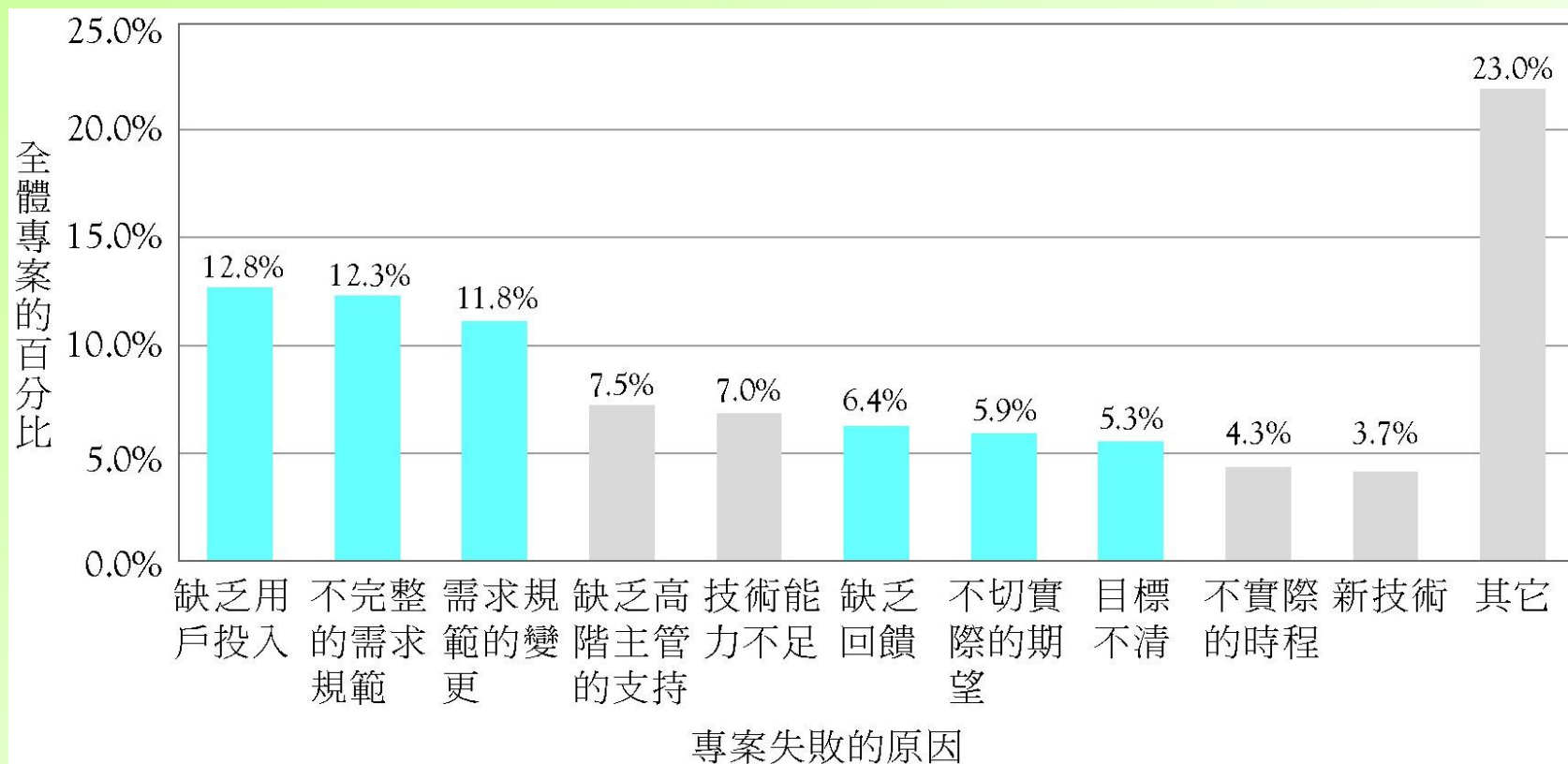
從「程式」到「系統」

- 軟體開發的另一個難題，是從單一程式到軟體系統過程中所造成複雜度的快速上升，其間需要包含不同的活動與技能，使得軟體開發必須面對多樣性的挑戰
- 系統一般是由許多子系統構成的，因此系統的架構與子系統間的協同作業一致性是很重要的
- 不可見性(invisibility)：軟體是執行在電腦硬體上，不像其他工程的產出是有實體的存在，因使軟體系統的規模估算、品質評估相對是較困難的
- 對程式與軟體系統的品質要求是有很大的差異
- 軟體生命週期包含軟體開發與維護，但通常軟體開發與維護不一定是同樣的人

從「個人」到「團隊」

- 一般軟體系統非由一個人單獨開發，而是由一個團隊來負責開發，因此如何**團隊與個人的管理**就很重要，但也較困難
- 不管是**開發產品**或**客製化**的軟體開發，產品/專案開發都會有**資源的限制**（工時/成本、時間）與品質的要求，因此如何達成產品/專案管理的目標（**如期、如本、如質**）就很重要了
- **PDCA管理循環**的議題：規畫、執行、查核監控、改善行動
- **團隊管理**的議題：組織與分工、領導統御、溝通協調、績效考核等等
- **專案管理**的議題：範疇管理、時間管理、成本管理、品質管理、人力資源管理、溝通管理、風險管理軟體等等

軟體專案失敗的原因



何謂工程

- 工程是一種將科學、數學、技術、知識及經驗應用於設計、建構和維護系統、產品或流程的專業領域
- 它的核心目標是利用有限的資源，以有效且經濟的方式解決實際問題，並創造出有價值的東西
- 簡單來說，工程是「動手解決問題」的學問，它不僅僅是理論，更強調實際應用和可行的解決方案

工程原則(Engineering Principle) (1/2)

- 原則一：完整詳實的標準作業程序

作業程序及步驟要定義的很明確與可行，並依照所制訂的程序來執行，並持續檢討與優化SOP

- 原則二：分解/模組化

將一個複雜的大需求或問題分解成階層式的小需求或小問題，並將問題的解決方案於以模組化，以利重複使用

- 原則三：遞增法

將上述階層式的分解結構圖依其重要性先完成較重要的部分，在確認沒問題後逐步遞增完成大系統的建置

- 原則四：優先順序

對事物或任務進行安排、分配或處理時，依照其相對重要性、緊急程度、截止日、可行性、不確定性、成本效益、複雜度來確定其處理或執行的優先級別或次序

工程原則(Engineering Principle) (2/2)

- 原則五：預視改變

在處理目前的問題時，需要想到未來問題與環境等是否會變動，亦即要有風險/不確定的意識，並能及時作好規劃

- 原則六：焦點分離

將一個複雜的問題分為不同的層次或構面來處理，而不同的層次或構面需要專注與聚焦在各自要處理的問題上

- 原則七：通用性

將一個特定問題的解決方法於以通用性，使其能適用在一般性問題的解決，達到再使用與擴大其效益

- 原則八：抽象化 (Abstraction)

將一個複雜的系統或問題經由概念化與簡化的過程，以讓使用者易於理解與使用，進而降低系統複雜度、提高效率或強化可維護性

工程有哪些重要的原則或準則？

- **系統化與紀律 (Systematic and Disciplined)**
 - ▶ 工程師在解決問題時，會採用系統化的方法。他們不會憑直覺做事，而是遵循一套有紀律的流程，例如定義問題、分析與設計、實施與測試、維護與改進
- **經濟與效率 (Economical and Efficient)**
 - ▶ 工程的核心原則之一是在有限的資源下，追求最大的效益。這意味著工程師必須同時考慮成本、時間與品質。
- **可靠性與安全性 (Reliability and Safety)**
 - ▶ 工程師設計的產品或系統必須具備高度的可靠性，確保在預期條件下能穩定運作，並且必須考量到安全性，避免對使用者或環境造成危害。在軟體開發中，這代表著軟體必須穩定、不容易當機，並且能夠保護使用者資料不被洩漏。
- **可維護性與可擴充性 (Maintainability and Scalability)**

工程的重要原則

不同工程領域會有些差異，但共同的核心原則包括：

- 需求導向：一切設計與建造都必須基於清楚定義的需求
- 系統化：採用結構化的方法分解問題，規劃流程，逐步落實
- 規格與標準化：遵循既定規範，確保品質一致性與相容性
- 品質保證：透過測試、檢驗、審查，確保產品符合需求並能長期可靠運作
- 成本效益：在有限資源下達成最佳效益，避免浪費
- 風險管理：預測潛在問題，設計備援與應對方案。
- 持續改進：隨著需求、技術、環境改變，不斷回饋與優化

工程的重要原則

- 系統性：有計畫、有步驟地解決問題
- 可衡量性：成果可被量化（如效能、成本、品質）
- 可重複性：流程可被複製並持續改善
- 風險控管：預測與防範失敗的可能性

軟體工程與管理重要理論或現象

● 專案管理三角理論

- ▶ 指在專案管理中，三個主要專案限制或變數-範疇、時間和資源之間存在的關係，這個理論幫助了解專案管理中的折衝和取捨，以及在限制內達成專案目標的挑戰

● 範疇/需求蔓延 (Scope Creep)

- ▶ 指在軟體開發過程中，使用者對軟體的需求範圍會不斷的變動、擴大或不清晰，而導致專案進度延誤、成本增加、品質下降等問題

● 漣漪效果 (Ripple Effect)

- ▶ 指對軟體系統的一個變更可能會引起相應的變更傳播到系統的其它部分，進而影響整個系統的功能、架構或其他方面，進而導致系統變得更複雜、難以維護、開發進度延遲，並增加軟體開發和維護的成本

軟體工程與管理其它理論/現象

- Brook's Law
- Pareto Principle
- 人月神話 (The Mythical Man-Month)
- 帕金森定律 (Parkinson's Law)
- 專案進度90% 症候群
- 學生症候群 (Student Syndrome)
- 技術債 (Technical Debt)
- 破窗效應 (Broken Windows Theory)
- 非我所創症候群 (Not Invented Here, NIH)

Software engineering fundamentals

- Some fundamental principles apply to **all types of software system**, irrespective of the development techniques used:
 - ▶ Systems should be developed using a **managed and understood development process**. Of course, different processes are used for different types of software.
 - ▶ **Dependability/Reliability/Usability** and **performance** are important for all types of system.
 - ▶ **Understanding and managing the software specification and requirements** (what the software should do) are important.
 - ▶ Where appropriate, you should **reuse** software that has already been developed rather than write new software.

SWEBOK 18 KA (知識領域)

**Guide to Software Engineering Body of Knowledge
(SWEBOK) V4.0a by IEEE Computer Society, 2024**

1. **Software Requirement**
2. **Software Architecture (V4)**
3. **Software Design**
4. **Software Construction**
5. **Software Testing**
6. **Software Engineering Operations (V4)**
7. **Software Maintenance**
8. **Software Configuration Management**
9. **Software Engineering Management**
10. **Software Engineering Process**

SWEBOK 18 知識領域KA

- 11. Software Engineering Models and Methods
- 12. Software Quality
- 13. **Software Security (V4)**
- 14. Software Engineering Professional Practices
- 15. Software Engineering Economic
- 16. Computing Foundations
- 17. Mathematical Foundations
- 18. Engineering Foundations

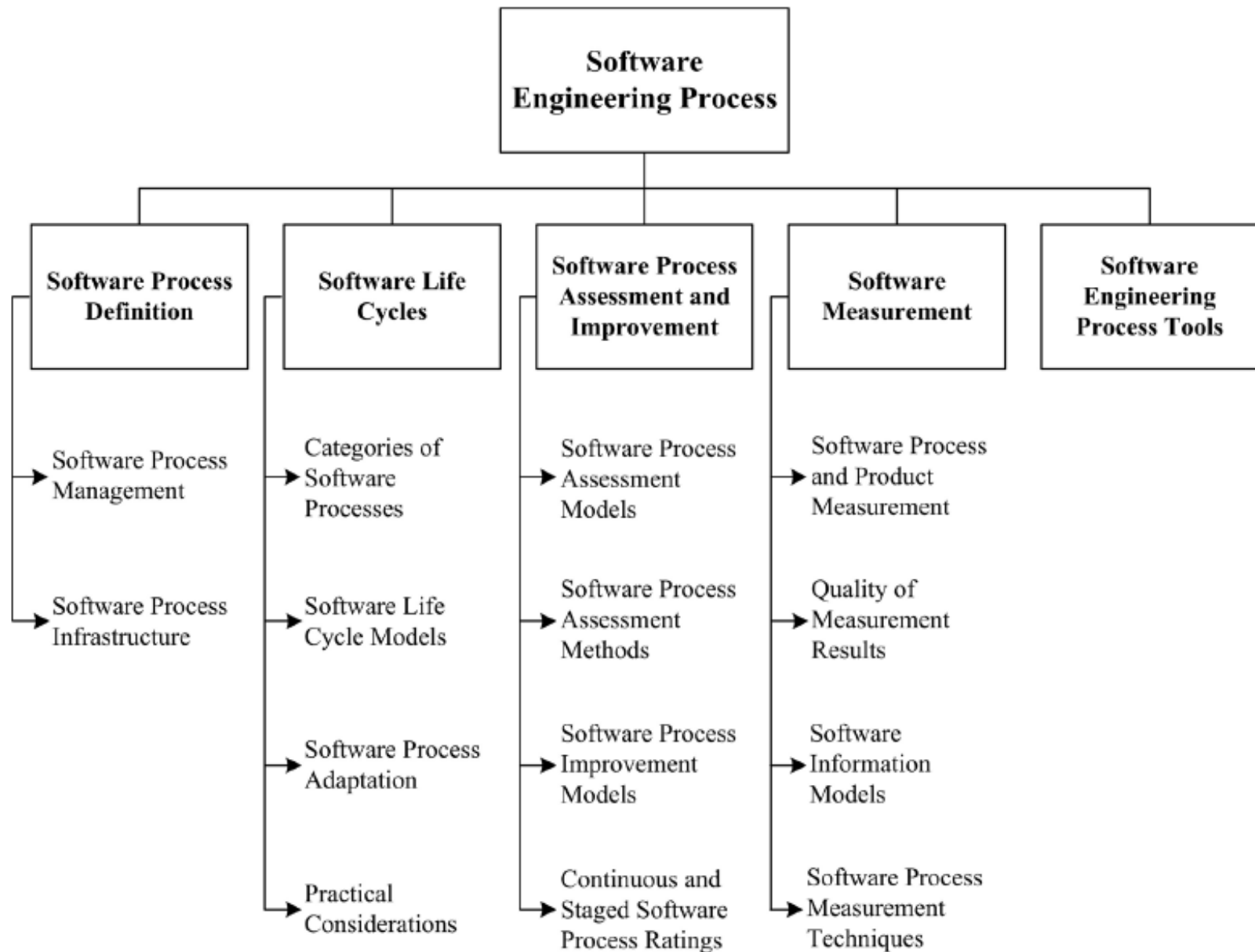


Figure 8.1. Breakdown of Topics for the Software Engineering Process KA

Who is the ISO?

- **International Organization for Standardization** is the world's largest developer of International Standards
- ISO is a network of the national standards institutes of 172 countries, **one member per country**
- ISO is a **non-governmental** organization that forms a bridge between the public and private sectors
 - Many of its member institutes are part of the governmental structure of their countries, or are mandated by their government
 - Other members have their roots uniquely in the private sector, having been set up by national partnerships of industry associations
- This enables ISO to reach a **consensus on solutions** that meet both the requirements of business and the broader needs of society

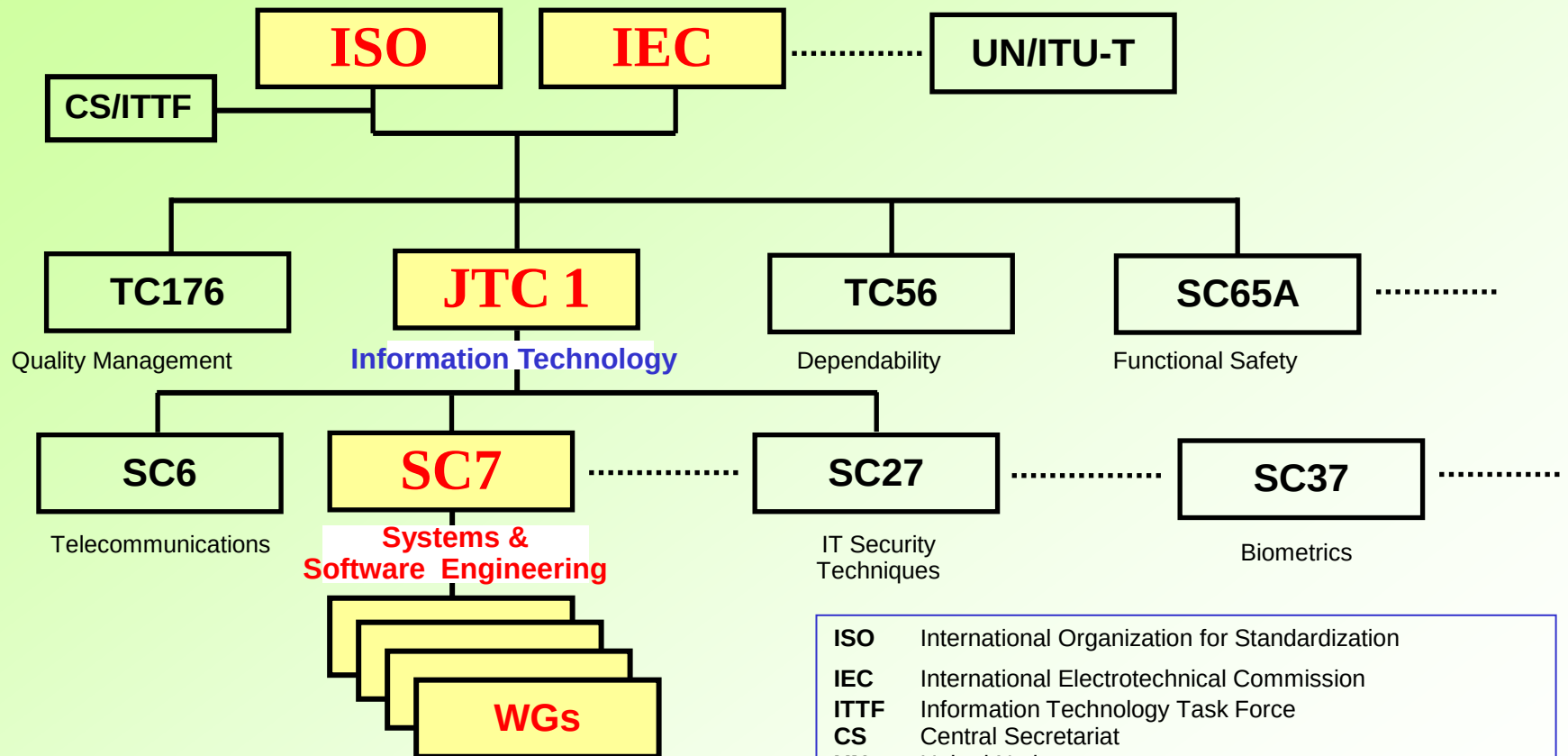
Who develops ISO standards

- ISO standards are developed by **technical committees**, (or subcommittees) comprising **experts** from the **industrial, academic, technical** and **business** sectors
- These experts may be joined by representatives of government agencies, consumer associations, non-governmental organizations and academic circles, etc.
- Experts participate as **national delegations**, chosen by the ISO national member body for the country concerned.

How ISO standards are developed

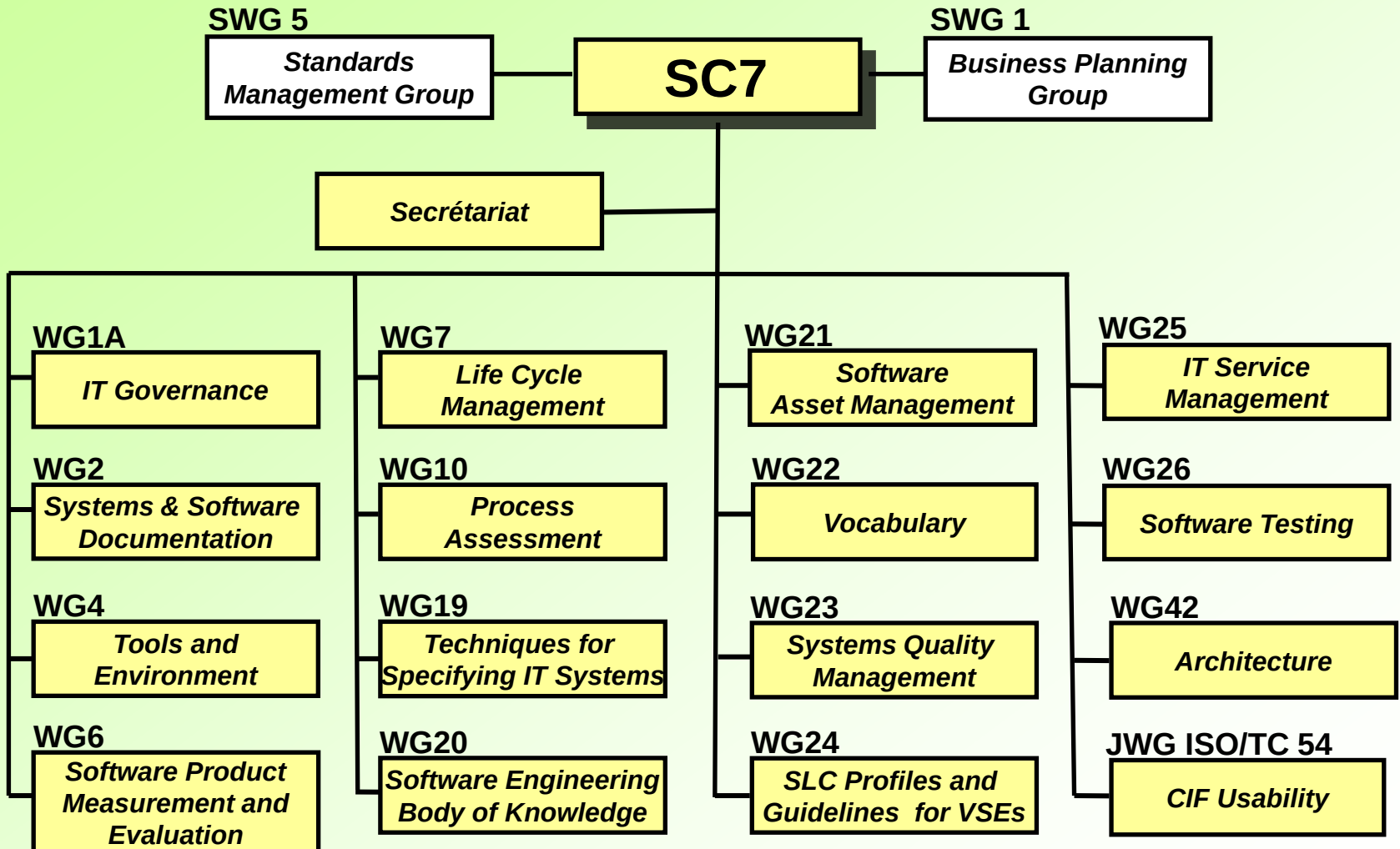
- The national delegations of experts of a committee meet to discuss, debate and argue until they **reach consensus** on a draft agreement
- The resulting document is circulated as a Draft International Standard (**DIS**) to all ISO's member bodies for voting and comment
- If the voting is in favor, the document, with eventual modifications, is circulated to the ISO members as a Final Draft International Standard (**FDIS**)
- The voting process is the key to consensus. If that's achieved then the draft is on its way to becoming an ISO standard. If agreement isn't reached then the draft will be modified further, and voted on again.
- From first proposal to final publication, developing a standard usually takes about 3 years.

ISO/IEC outline Structure

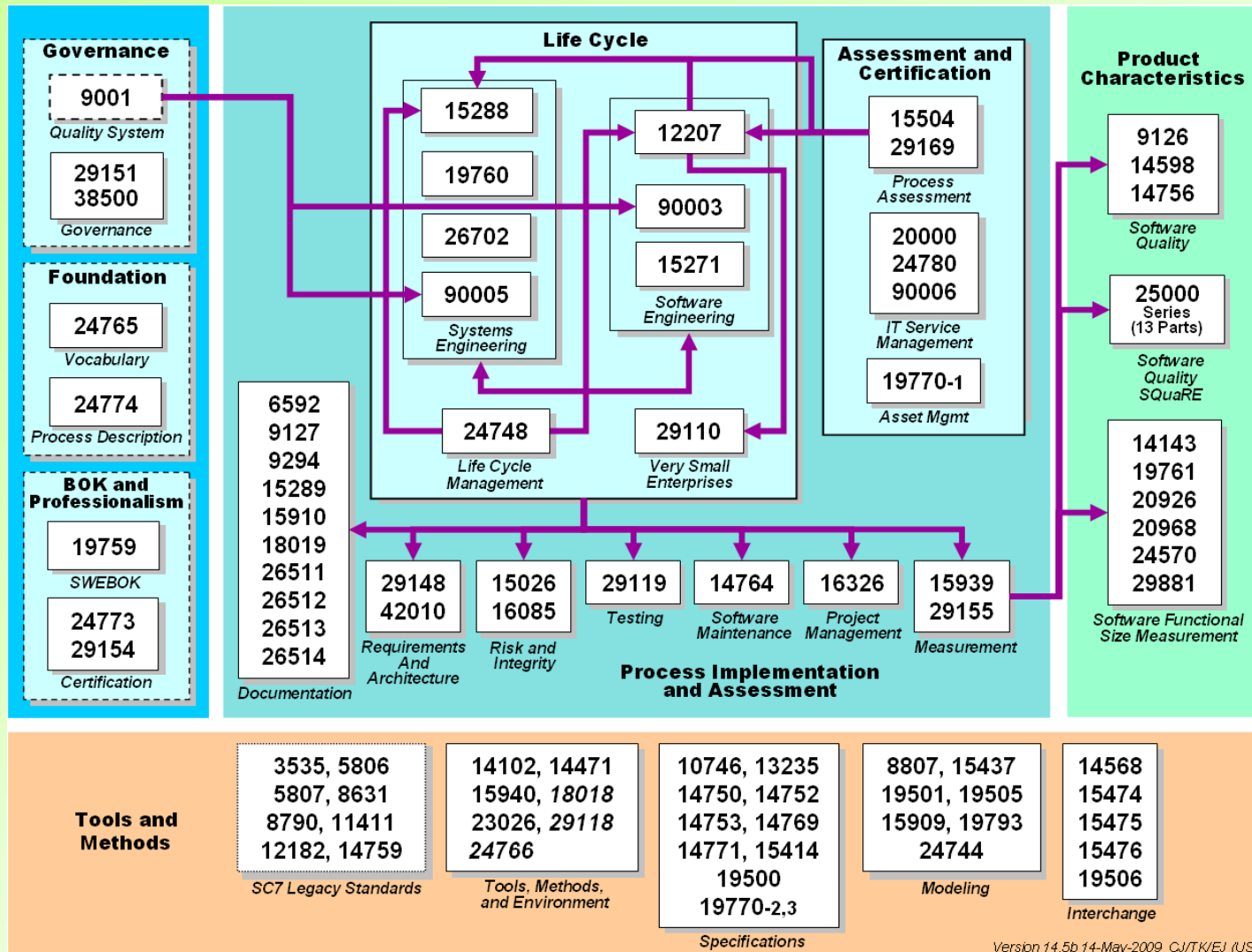


ISO	International Organization for Standardization
IEC	International Electrotechnical Commission
ITTF	Information Technology Task Force
CS	Central Secretariat
UN	United Nations
ITU-T	International Telecommunications Union
TC	Technical Committee
SC	Sub Committee
JTC	Joint Technical Committee
WG	Working Group

SC7 Structure



JTC 1 SC7 Standards Collection



ISO Software Engineering Standards

- www.iso.org
- Standards by ISO/IEC JTC 1 **Information technology**
- ISO/IEC JTC 1/SC 7 **Software and systems engineering**
- How many **published standards** under ISO/IEC JTC 1/SC 7? How many standards **under development**?
- How many published standards under ISO by ICS 35.080 **Software** (including development, documentation and use)?

IEEE Software Engineering Standards

- <https://library.ntust.edu.tw/home.php>
- 常用資料庫 IEEE (IEL)
- <https://ieeexplore-ieee-org.ezproxy.lib.ntust.edu.tw/Xplore/home.jsp>
- Browse Standards
- By collection => Information Technology => Software
How many active standards?
What are the newest standards?
- By ICS code
How many active standards of ICS code 35.08 Software ?
What is the first standard sorted by standard number?

國內軟體公司較常採用與取得認證的標準

- ISO 9001:2015 Quality management systems — Requirements
- ISO/IEC 27001 Information security management systems — Requirements
- ISO/IEC 27701:2022 Security techniques — Extension to ISO/IEC 27001 and ISO/IEC 27002 for **privacy information management** — Requirements and guidelines
- ISO/IEC 27017:2015 Information technology — Security techniques — Code of practice for information security controls based on ISO/IEC 27002 for cloud services
- ISO/IEC 20000-1:2011 Information technology — Service management — Part 1: Service management system requirements [This standard was last reviewed and confirmed in 2021. Therefore this version remains current]
- ISO 14001:2015 Environmental management systems — Requirements with guidance for use

國內軟體公司較常採用與取得認證的標準

- ISO 22301:2019 Security and resilience — Business continuity management systems — Requirements
- ISO 31000-2018 Risk management — Guidelines
- ISA/IEC 62443 工業自動化及控制系統(Industrial Automation and Control System)網通安全(Cybersecurity)
個人認證

小結 (1/2)

- 軟體開發是一項高複雜、高難度、高風險的活動，由於高失敗率，故有「**軟體危機**」一詞
- 軟體工程發展至今，依然存在著許多問題，**尚未能有標準的成功法則**，所以學習軟體工程這門科學，不能一知半解，否則或許差之毫釐，卻失之千里。
- 不只要學習**Know-what**與**Know-how**，還必須要瞭解**Know-why**。應透過**思辨**的過程，深入瞭解各種方法背後的**假設與邏輯**，以避免掉入各種軟體開發陷阱。
- 軟體開發採用的技術、方法與管理需要考慮到不同的軟體類型，**沒有最好的技術、方法**，只有**最適合的**。
- 工程師們須培養**多樣化的技能**，尤其是瞭解與分析用戶需求的能力。此外，在**團隊合作的過程中**，**溝通與管理的技能**也不可或缺的；在可預見的未來，**人的素質**依然決定軟體產品的品質。

小結(2/2)

- **用整體觀點**看待軟體開發：許多專案之所以遭遇到困難，是因為消耗太多資源在專案的某一方面，忽略了其它應注意而未注意的事項，以致於後續發生問題時，再回頭來彌補或救火，卻事倍功半甚至無法達到目的。
- **建立工程與管理的基礎**：知道理論不表示就能夠做好事情，除非具備基本的**實務訓練**，否則更多作為最後可能只是空談。對於軟體工程的各項實務，應設法學習、瞭解，並導入軟體開發專案之中。
- **避免典型疏失**：軟體專案牽涉到非常多的決策；成功的專案，意味著沒有在開發之路上，犯下一些基本的錯誤。軟體開發的過程中藏有許多陷阱，而大部分的疏失都起因於一些直覺上看起來合理的理由。認識這些疏失並儘量避開，是軟體專案開發成功的先決條件。